

Task 2 README — n8n API Integration Workflow

Workflow: GitHub AI Repos Morning Brief | Author: Candidate | Date: 25 May 2026

1. Workflow Overview

The **Morning Brief** workflow runs every hour, fetches the top AI-related GitHub repositories, enriches the results, applies a threshold branch, and posts a formatted digest to a Discord channel. A separate Error Trigger catches any unhandled failure and routes it to the same channel as an alert.

2. Node-by-Node Flow

Step	Node	Purpose
1	Schedule Trigger (Every 1h)	Fires the workflow on the hour, every hour. Can be replaced with a Webhook trigger for manual testing.
2	GitHub Search – AI Repos	GET <code>https://api.github.com/search/repositories?q=topic:ai&sort=stars</code> — returns up to 20 repos sorted by stars. Uses a GitHub Personal Access Token stored in n8n Credentials (<code>httpHeaderAuth</code>), not hardcoded.
3	Transform – Top 5 Repos	Code node: sorts by stars descending, slices to top 5, maps to a clean schema (name, description, stars, forks, language, url). Sets <code>highStars=true</code> when stars > 1 000. Guards against empty/error responses.
4	IF – Stars > 1 000	Routes each repo: TRUE branch (hot) or FALSE branch (regular). Threshold of 1 000 was chosen because it reliably separates well-established projects from new/niche ones.
5a	Enrich – Fetch Repo Details (Hot)	Second GitHub API call: GET <code>/repos/{owner}/{repo}</code> — retrieves <code>open_issues_count</code> , <code>watchers_count</code> , and <code>licence</code> . Only fires for repos that passed the threshold.
5b	Enrich – Fetch README (Regular)	Second GitHub API call: GET <code>/repos/{owner}/{repo}/readme</code> — checks whether a README exists and logs its size in bytes. Fires for below-threshold repos.
6a/b	Merge Hot / Merge Regular Enrichment	Code nodes that combine enrichment data with the original repo object and apply a badge (■ hot, ■ regular).
7	Merge All Results	n8n Merge node (append mode) re-unifies the two branches into a single list before message building.
8	Build Discord Message	Code node that formats all 5 repos into a Markdown-style Discord message (max 2 000 chars). Includes rank, badge, stars, forks, language, and tier label.
9	Send to Discord Webhook	HTTP POST to the Discord Incoming Webhook URL stored in n8n Credentials. <code>continueOnFail=true</code> so a Discord outage doesn't crash the workflow silently.
E1	Error Trigger	Catches any unhandled exception from any node in this workflow.
E2	Format Error Message	Builds a concise alert with workflow name, last node, and error message.
E3	Send Error to Discord	Posts the error alert to the same Discord channel so failures are never silent.

3. APIs Used & Why

GitHub REST API v3 (Search + Repo endpoints): Free with a Personal Access Token (5 000 req/h authenticated vs 60/h unauthenticated). The search endpoint provides a rich, sortable dataset. The repo endpoint gives secondary enrichment data (issues, licence) without needing a separate third-party API. Chosen because it is well-documented, reliable, and directly relevant to a dev-tooling team.

Discord Incoming Webhook: Zero-cost, no bot setup needed — paste a URL and POST JSON. Delivers formatted Markdown messages to any channel. Chosen over email because it is instant, team-visible, and supports rich formatting without SMTP configuration.

4. What the Transformation Does

The Code node (step 3) receives the raw GitHub search response, which contains up to 20 repo objects with 40+ fields each. It:

- Sorts repos by stargazers_count descending (GitHub's default sort can differ).
- Slices to the top 5, discarding the rest.
- Projects each repo to 9 fields: rank, name, description (truncated to 120 chars), stars, forks, language, url, api_url, pushed_at.
- Sets highStars = stars > 1 000 as a boolean flag consumed by the downstream IF node.
- Returns early with an error object if the API returned an error or empty list, preventing downstream nodes from processing junk data.

5. How the Error Path Behaves

Error handling operates at two levels:

continueOnFail = true on every HTTP node: if GitHub or Discord is temporarily unreachable, the node passes an error object downstream rather than halting execution. The Transform and Merge Code nodes detect this pattern (checking for json.error or json.message) and short-circuit gracefully.

Error Trigger workflow-level handler: any truly unhandled exception (e.g. a Code node throws, or n8n itself encounters an issue) is caught by the Error Trigger. It formats a structured alert containing the workflow name, the last node that ran, the error message, and a timestamp — then posts it to Discord. This means the team is notified within seconds of any silent failure, satisfying the 'must NOT crash silently' requirement.

6. Credentials & Security

No secrets are hardcoded in any node. Two credential entries are required in n8n:

- **GitHub Token Header:** Type: HTTP Header Auth. Header name: Authorization. Value: Bearer <your-github-pat>. Used by the two GitHub HTTP nodes.
- **Discord Webhook URL:** Type: HTTP Query Auth (URL stored as credential). Used by all three Discord output nodes.